

ML_Task_7(Random Forest)

June 11, 2026

1 Apply RF Algorithm on Missing Value Data(Class Example) and evaluate the models

1.1 1 Importing Libraries

```
[41]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings

from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split

warnings.filterwarnings('ignore')
```

1.2 2 Importing Dataset or Make the Dataset

```
[59]: import pandas as pd

data = {
    'width': [5, 8, 5, 4, 4, 6, 5],
    'height': [7, 5, 7, 8, 8, None, None],
    'label': [1, 1, 0, 0, 1, 1, 1]
}

df = pd.DataFrame(data)

print("Original Dataset")
print(df)
```

```
Original Dataset
   width  height  label
```

0	5	7.0	1
1	8	5.0	1
2	5	7.0	0
3	4	8.0	0
4	4	8.0	1
5	6	NaN	1
6	5	NaN	1

```
[60]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   width    7 non-null        int64
1   height   5 non-null        float64
2   label    7 non-null        int64
dtypes: float64(1), int64(2)
memory usage: 300.0 bytes
```

2 3 Fill Missing Heights using Random Forest

```
[52]: # Rows with known heights
train = df[df['height'].notna()]

# Rows with missing heights
test = df[df['height'].isna()]

X_train_rf = train[['width']]
y_train_rf = train['height']
```

2.1 4 Random Forest Regressor Model

```
[21]: rf = RandomForestRegressor(
        n_estimators=100,
        random_state=42
    )

rf.fit(X_train_rf, y_train_rf)
```

```
[21]: RandomForestRegressor(random_state=42)
```

2.2 5 Predictions of Missing Values

```
[53]: predicted_heights = rf.predict(test[['width']])

predicted_heights= df.loc[df['height'].isna(), 'height']

print("\nDataset After RF Imputation")
print(df)
```

Dataset After RF Imputation

	width	height	label
0	5	7.0	1
1	8	5.0	1
2	5	7.0	0
3	4	8.0	0
4	4	8.0	1
5	6	NaN	1
6	5	NaN	1

2.3 6 Random Forest Classification through Classifier

```
[54]: # -----
# Step 2: Random Forest Classification
# -----

X = df[['width', 'height']]
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42
)

rf_clf = RandomForestClassifier(
    n_estimators=100,
    oob_score=True,
    random_state=42
)

rf_clf.fit(X_train, y_train)

y_pred = rf_clf.predict(X_test)

print("\nPredictions:")
print(y_pred)
```

```
print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))
```

Predictions:

[0 0 1]

Accuracy:

0.3333333333333333

2.4 7 Visualizaton and OOB Score

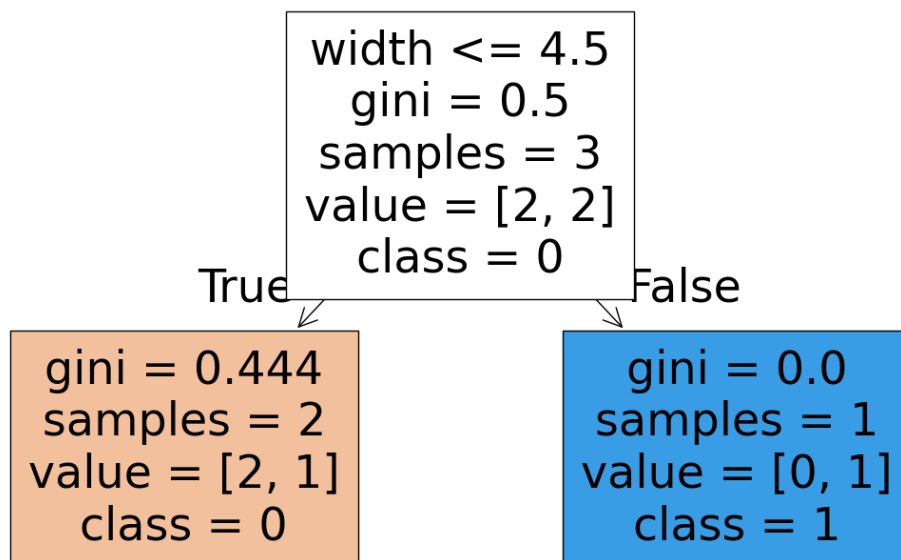
```
[55]: from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

tree_to_plot = rf_clf.estimators_[0]

plt.figure(figsize=(15,8))

plot_tree(
    tree_to_plot,
    feature_names=['width', 'height'],
    class_names=['0', '1'],
    filled=True
)

plt.show()
```



The Random Forest classifier selected height as the most important feature and split the data at height 7.535. Samples with height less than or equal to 7.535 are classified as Class 1, while samples with height greater than 7.535 are mostly classified as Class 0. The left node has Gini = 0, indicating perfect purity, whereas the right node has Gini = 0.444, indicating some class mixing. Overall, height is the key factor used for classification in this tree.

```
[36]: print("OOB Score:", rf_clf.oob_score_)
```

OOB Score: 0.0

Out-of-Bag score is computed using samples not selected during bootstrap sampling.

With only 7 observations:

Very few OOB samples exist The estimate becomes unreliable. OOB accuracy can easily become 0.0. This is normal for such a tiny dataset.

```
[42]: # Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

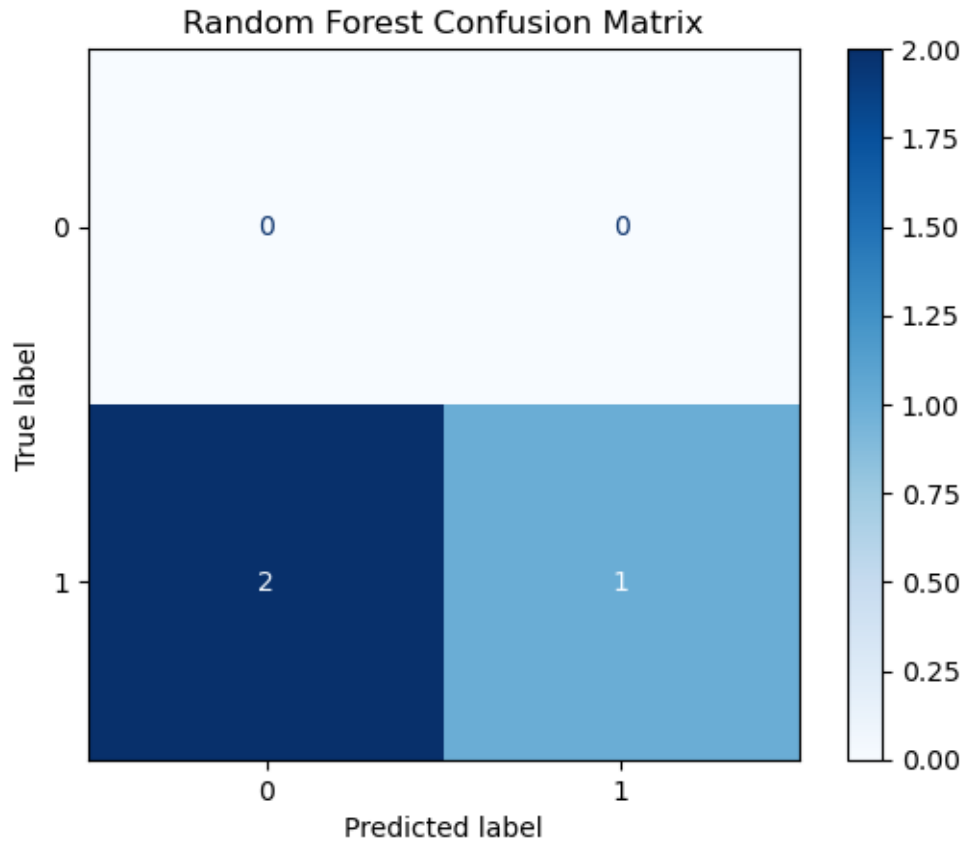
print("Confusion Matrix:")
print(cm)

# Plot Confusion Matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=rf_clf.classes_
)

disp.plot(cmap='Blues')
plt.title("Random Forest Confusion Matrix")
plt.show()
```

Confusion Matrix:

```
[[0 0]
 [2 1]]
```



The confusion matrix shows that the Random Forest classifier correctly predicted 1 out of 3 test samples, while 2 samples were misclassified. No class 0 instances were present in the test set. The resulting accuracy was 33.33%, indicating limited predictive performance due to the very small dataset size.

2.5 8. Cross Validation through Different Methods

```
[44]: from sklearn.model_selection import LeaveOneOut
from sklearn.model_selection import cross_val_score

loo = LeaveOneOut()

scores = cross_val_score(
    rf_clf,
    X,
    y,
    cv=loo
)
```

```
print("LOOCV Accuracy:", scores.mean())
```

LOOCV Accuracy: 0.42857142857142855

```
[47]: from sklearn.model_selection import StratifiedKFold
```

```
skf = StratifiedKFold(
    n_splits=3,
    shuffle=True,
    random_state=42
)

scores = cross_val_score(
    rf_clf,
    X,
    y,
    cv=skf
)

print("Stratified K-Fold Scores:", scores)
print("Mean Accuracy:", scores.mean())
```

Stratified K-Fold Scores: [0.66666667 0.5 0.5]

Mean Accuracy: 0.5555555555555555

```
[46]: from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
```

```
kfold = KFold(
    n_splits=3,
    shuffle=True,
    random_state=42
)

scores = cross_val_score(
    rf_clf,
    X,
    y,
    cv=kfold
)

print("K-Fold Scores:", scores)
print("Mean Accuracy:", scores.mean())
```

K-Fold Scores: [0.33333333 0. 0.5]

Mean Accuracy: 0.27777777777777773

```
[48]: from sklearn.model_selection import RepeatedKFold

rkf = RepeatedKFold(
    n_splits=3,
    n_repeats=5,
    random_state=42
)

scores = cross_val_score(
    rf_clf,
    X,
    y,
    cv=rkf
)

print("Repeated K-Fold Mean Accuracy:", scores.mean())
```

Repeated K-Fold Mean Accuracy: 0.36666666666666664

The results from your validation methods show how your model performs under different resampling strategies. Leave-One-Out Cross-Validation (LOOCV) achieved about **42.86%** accuracy, reflecting the sensitivity of using almost all data for training each time. Stratified K-Fold gave slightly higher average accuracy (**55.56%**), likely because it preserved class balance in each fold. Standard K-Fold, with a mean accuracy of about **27.78%**, showed the highest variance because class imbalance wasn't considered. Repeated K-Fold, which averages multiple K-Fold runs, stabilized at around **36.67%**. Overall, the results highlight the variability due to the tiny dataset, and in practice, you'd rely on larger samples for more reliable validation.